Google DeepMind



stage0: a minimal firmware for (confidential) virtual machines

Andri Saar
Google DeepMind, Project Oak

Kevin Hui
Meta, Private Compute Platform

Oak is a software platform for building distributed systems providing externally verifiable (or falsifiable) claims about system behaviors in a transparent way.

https://github.com/project-oak/oak

Requirements for Oak firmware

Run on QEMU (and compatible) VMMs

Primary goal is to support `pc` and `microvm` machine types on QEMU (and VMMs close enough), with `q35` a secondary target.

The firmware must set up the machine enough to be usable by Linux; no bells and whistles.

Support AMD SEV-SNP and Intel TDX

We want to run in encrypted virtual machines, and thus we need to support the two x86 enclave technologies.

Include only code that is absolutely necessary

We don't need anything fancy: do minimal platform initialization, load the kernel and initrd, hand reins over to Linux, and that's it.

We don't need support for network, block devices, graphics, netboot, ...

Boot Linux(-compatible) kernels

The goal is to get Linux up and running. Compatibility with other operating systems is not a requirement.

(Sorry, Windows.)

Existing firmware choices

Firmwareless

In principle the ideal solution: the most secure code is code that does not exist.

This is what Cloud Hypervisor or crosvm do by default.

- Requires VMM to set up the machine beyond what QEMU does
- Compatibility questions around TDX

OVMF

UEFI firmware built from TianoCore EDK2.

- Active development by participants across the whole industry
- Can be considered the “default” firmware
- Supports many platforms, OSes and features
- Provides EFI runtime services
- Complex code base, leading to potential security risks

SeaBIOS

Legacy BIOS implementation; still used by default by QEMU.

- Does not support SEV-SNP or TDX, so would need enlightenment work
- Mainly provides legacy BIOS services
- Simpler than OVMF

Oak stage0 firmware

Written in Rust, with assembly only where strictly necessary.

- Supports SEV-SNP and Intel TDX
- Uses `fw_cfg` to load the kernel and initrd along with ACPI tables from the hypervisor
  - we're not interpreting AML
- Performs basic PCI bus configuration
- Under SEV-ES and above, initializes and parks APs for the kernel to initialize
- Builds attestation evidence for all the software that was loaded
  - under SEV-SNP, we reserve a page in E820 table for DICE chain and destroy VMPCK0 so no more attestations may be obtained

...and then hands reins over to the Linux kernel.

```
stage0 INFO: starting...
stage0 INFO: Enabled SEV features: SevStatus(SEV_ENABLED | SEV_ES_ENABLED | SNP_ACTIVE)
stage0 INFO: starting SEV-SNP memory validation
stage0 INFO: SEV-SNP memory validation complete.
stage0 INFO:   Validated using 2 MiB pages: 4095
stage0 INFO:   Validated using 4 KiB pages: 272
stage0 INFO:   Valid state not updated: 0
stage0 INFO:   RMP page size mismatch errors (fallback to 4K): 0
stage0 DEBUG: Kernel cmdline: console=ttyS0 panic=-1 earlycon=uart,io,0x3F8 brd.rd_nr=1 brd.rd_size=3072…
stage0 DEBUG: Kernel image size 5971968
stage0 DEBUG: Kernel image start address 0x0000000002000000
stage0 DEBUG: Kernel entry point 0x0000000002000200
stage0 INFO: PCI: using windows PciWindows { pci_window_16: 49152..65535, pci_window_32: 3221225472..422…
stage0 DEBUG: Found PCI device: 00:00.0, 8086:1237, class: 0600
stage0 DEBUG: Found PCI device: 00:01.0, 8086:7110, class: 0601
stage0 DEBUG: Found PCI device: 00:01.3, 8086:7113, class: 0680
stage0 DEBUG: Found PCI device: 00:03.0, 1af4:1000, class: 0200
stage0 INFO: Using xAPIC for AP initialization.
stage0 INFO: Expected number of APs: 0, started number of APs: 0
stage0 DEBUG: Initial RAM disk size 4439549
stage0 DEBUG: Initial RAM disk address 0x000000003fac4000
stage0 DEBUG: Kernel image digest: sha2-256:f9d0584247b46cc234a862aa8cd08765b38405022253a78b9af189c4cedb…
stage0 DEBUG: Kernel setup data digest: sha2-256:5a5cd42b9471aadf96dc2d2407d9dcbb1165c28c7eea1a0b4e9c72c…
stage0 DEBUG: Kernel command-line: console=ttyS0 panic=-1 earlycon=uart,io,0x3F8 brd.rd_nr=1 brd.rd_size…
stage0 DEBUG: Initial RAM disk digest: sha2-256:a8b5371594bfc2944fe6263148f6ef289821bf2e581b1ed976cc2f8…
stage0 DEBUG: ACPI table generation digest: sha2-256:a70dbecea0605c60e7bf16a114e4a3210c133517d304c776f4…
stage0 DEBUG: E820 table digest: sha2-256:79ae6053b1afd95db05643d3676af49fa7ea30c5b3579d090be9a18ae7030…
stage0 DEBUG: Event digest: sha2-256:2d5439f3d29572e510e95a6f5e5aa1a85b9cc70ffd3355397bcfe3cf6829b04f
stage0 DEBUG: Transparent event digest: sha2-256:48710fc8644073e90f2bbb252a9df65528ec876555285fc122af95…
stage0 INFO: jumping to kernel at 0x0000000002000200

Linux version 6.12.25 (user@host) (gcc (GCC) 14.2.1 20250322, GNU ld (GNU Binutils) 2.44) #1-NixOS SMP …
Command line: console=ttyS0 panic=-1 earlycon=uart,io,0x3F8 brd.rd_nr=1 brd.rd_size=3072000
brd.max_part=1
    ip=10.0.2.15:::255.255.255.0::eth0:off net.ifnames=0 quiet -- --oak-dice=0x15000
    --oak-event-log=0x16000 --oak-dice-length=16384
```

Examples of issues we've run into

Moving target from both sides

QEMU and OVMF/SeaBIOS development often happens in tandem; we're out of the tree and off the beaten path, so we often have to reverse engineer from existing implementation what the expected behaviour is.

There is often no clear specification as to what and how things should be implemented, both from the Linux side of things (what *is* the minimum the kernel needs?) and the hardware side.

Expectation of EFI used for booting

Because OVMF is the only (popular) firmware supporting SEV-SNP and TDX, it's easy to assume that when it works with OVMF, it's correct.

Example: https://lkml.org/lkml/2024/2/13/1227

Features like unaccepted memory implementation in kernel implementation assumes EFI boot services are available in bootloader, making the feature unusable with non-EFI bootloaders.

Examples of Oak stage0 users at Google

Private AI Compute

https://services.google.com/fh/files/misc/private_ai_compute_technical_brief.pdf

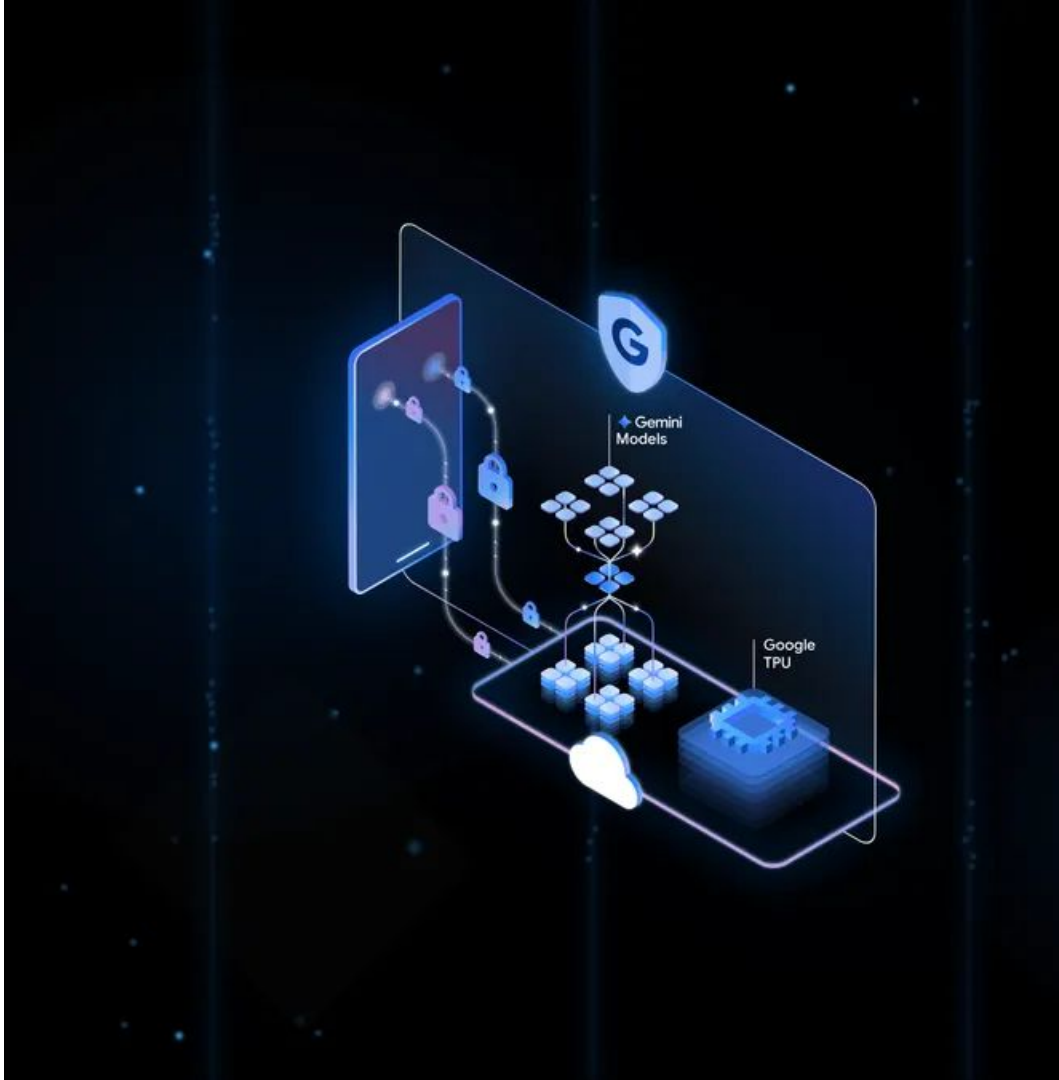
Confidential Federated Analytics

https://research.google/blog/discovering-new-words-with-confidential-federated-analytics/

Chrome Cloud Authenticator

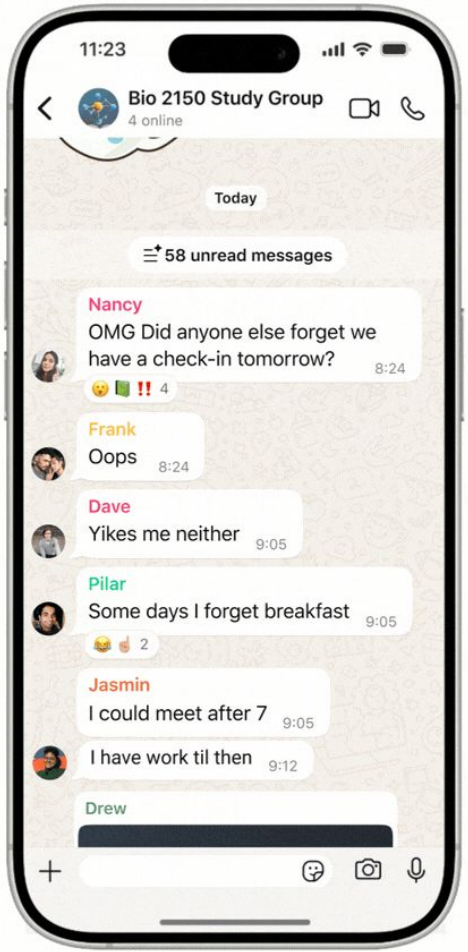
https://chromium.googlesource.com/chromium/src/+/HEAD/device/fido/enclave

https://chromium.googlesource.com/chromium/src/+/HEAD/third_party/cloud_authenticator/



Meta Private Processing & stage0

- Meta Private Processing CVMs uses stage0 for security and simplicity
- Embedded known ACPI table hashes to remove AML attack vector
  - Planned ACPI table generation to remove this issue
- Reproducible bootloader provides strong code provenance
  - Hoping to leverage a stage0 binary used by both Google and Meta



Making stage0 more OSS compatible

- First major consumer of stage0 outside of Google
  - Natural friction between Google/Meta infra
  - Meta uses QEMU + stage0 + vanilla kernel + NVIDIA GPU stack
  - Aiming for long term support of OSS stack for CVMs
- Upstreaming patches
  - Kernel-hashes support
  - CVE fixes
  - ACPI table generation/validation
  - Optional DICE attestation

Experimenting with Unaccepted Memory

- Stage0 proactively PVALIDATEs all memory pages at boot
  - Unaccepted memory changes amortize these costs
- Problem: UEFI assumptions baked into the implementation
  - allocate_unaccepted_bitmap() makes a lot of EFI-related calls
  - Initially tried hacking with dynamic allocation, but memory allocator isn't initialized at this point
  - Experimented with a minimal EFI implementation but abandoned the approach after a few crashes from missing EFI table info
- Looking for suggestions on how to avoid replicating UEFI while attaining unaccepted memory benefits
  - Theoretically E820 changes should suffice to leverage unaccepted memory

We still believe there is a need for a minimal firmware for virtual machines, particularly targeting confidential computing use cases.

Some future plans for stage0

Google:

- SVSM support for SEV-SNP
- Explore ARM CCA support
- Support for Cloud Hypervisor

Meta:

- Explore ACPI table generation
  - Attested in bootloader logic, avoid trusting hypervisor ACPI tables
- Add unaccepted memory support
- Continue upstreaming Meta patches to stage0 to unify both companies' stage0 binaries
- Explore transparent hugepage support for CVMs

As always, other interested parties are welcome to participate.

Some future plans for stage0

Google

- SVSM support for SEV-SNP
- Explore ARM CCA support
- Support for Cloud Hypervisor

Meta

- Explore ACPI table generation
  - Attested in bootloader logic, avoid trusting hypervisor ACPI tables
- Add unaccepted memory support
- Continue upstreaming Meta patches to stage0 to unify both companies' stage0 binaries
- Explore transparent hugepage support for CVMs

As always, other interested parties are welcome to participate.



Thank you.